



BITS Pilani

Pilani | Dubai | Goa | Hyderabad

AIML for Robotics

Module 2 – Sensors, Measurement, Model

Session Faculty:

Raja vadhana Prabhakar

The course owner and the authors of this course deck, Prof. Raja vadhana and Dr.Sangeetha, are gratefully acknowledging all authors who made their course materials freely available online.

☐ Introduction

- ☐ Types of Robots
- ☐ Mobile Robots – Use case of Programmable solutions

☐ Components of Programmable Robotic System

- ☐ Perception
- ☐ Representation
- ☐ Motion

☐ Localization

- ☐ Taxonomy
- ☐ Types

☐ Mapping

- ☐ Feature Based
- ☐ Grid Based

☐ Path Planning

- ☐ Graph Based
- ☐ Sample Based
- ☐ Evolutional Techniques

☐ Human – Robot Interaction

- ☐ Ethics
- ☐ Introduction to Visual Bot
- ☐ Introduction to Chat Bot

Sensor Measurement Model

Recap

1. Initial Belief
2. Control Update
3. Measurement Update

X_{t-1}	U_t	$P(X_t = \text{is_open})$	$P(X_t = \text{is_closed})$
is_open	push	1	0
is_closed	Push	0.8	0.2
is_open	do_nothing	1	0
is_closed	do_nothing	0	1

SENSOR MEASUREMENT MODEL : $P(Z_t | X_t)$

X_t	$P(Z_t = \text{sense_open})$	$P(Z_t = \text{sense_closed})$
is_open	0.6	0.4
is_closed	0.2	0.8

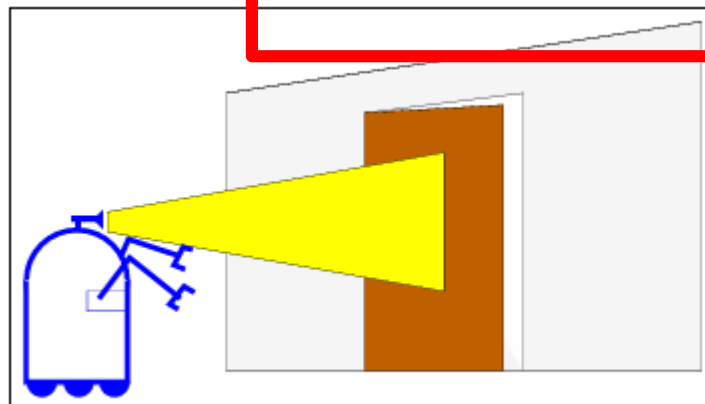
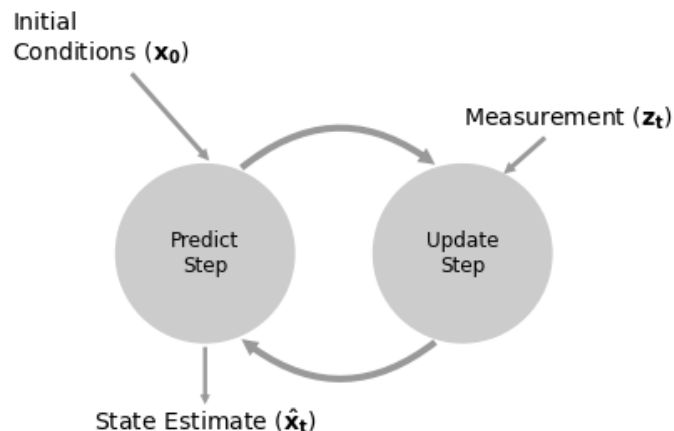
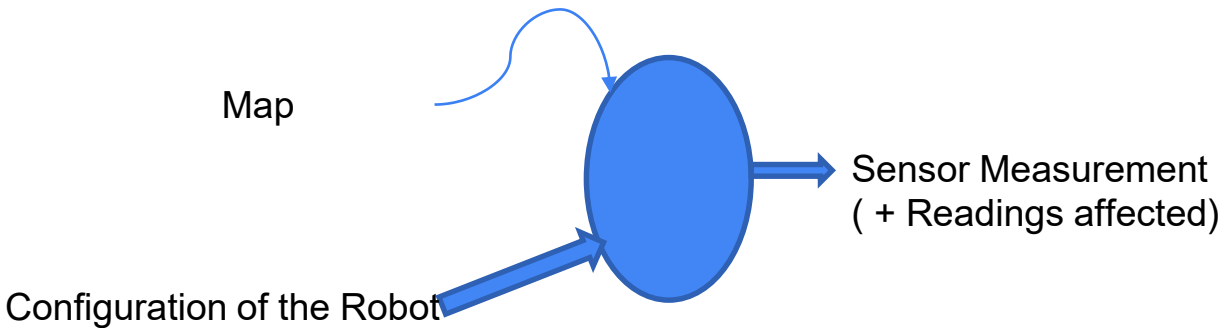


Figure 2.2 A mobile robot estimating the state of a door.

Inference :

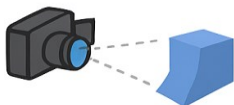
The robot believes that with 0.983 probability the door is open

Sensor Model



Analogy: Different Ways to 'Understand a Photo'

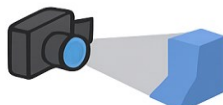
Beam Model



Examine every pixel's brightness and geometry

- highly detailed but slow

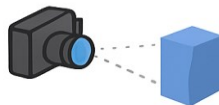
Likelihood Field



Check how 'close' the picture is to a known shape

- fast but approximate

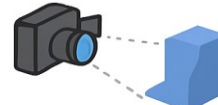
Correlation Model



Compare overall similarity between images

- robust but less precise

Feature-Based Model



Focus on key landmarks (eyes, corners, edges)

- efficient and abstract

Overview of Control Architectures

Architectural Paradigm



Traditional Robotics

Focuses on physical sensor integration and mechanical execution within the control architecture.



AIML for Robotics

Emphasizes organizational logic, algorithmic decision-making, and high-level behavioral subsumption.

Architecture Selection

innovate

achieve

lead

REACTIVE (Mandatory)



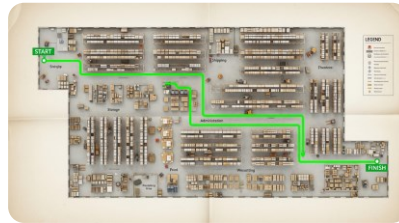
Mobile Robot Scenario:

Vacuuming: Immediate response to bumper contact is critical to prevent furniture damage.

CyberBot Scenario:

Logic: Safety-critical interrupt loops where code must halt execution instantly upon error detection.

DELIBERATIVE (Preferred)



Mobile Robot Scenario:

Pathfinding: Calculating the optimal path through a static warehouse for efficiency.

CyberBot Scenario:

Logic: Complex optimization algorithms solving for global minimums in data processing tasks.

HYBRID (Mandatory)



Mobile Robot Scenario:

Autonomous Delivery: Needs fast reflexes for pedestrians (Reactive) + high-level route planning (Deliberative).

CyberBot Scenario:

Logic: Real-time data streaming paired with background analytical processing.

HIERARCHICAL (Preferred)



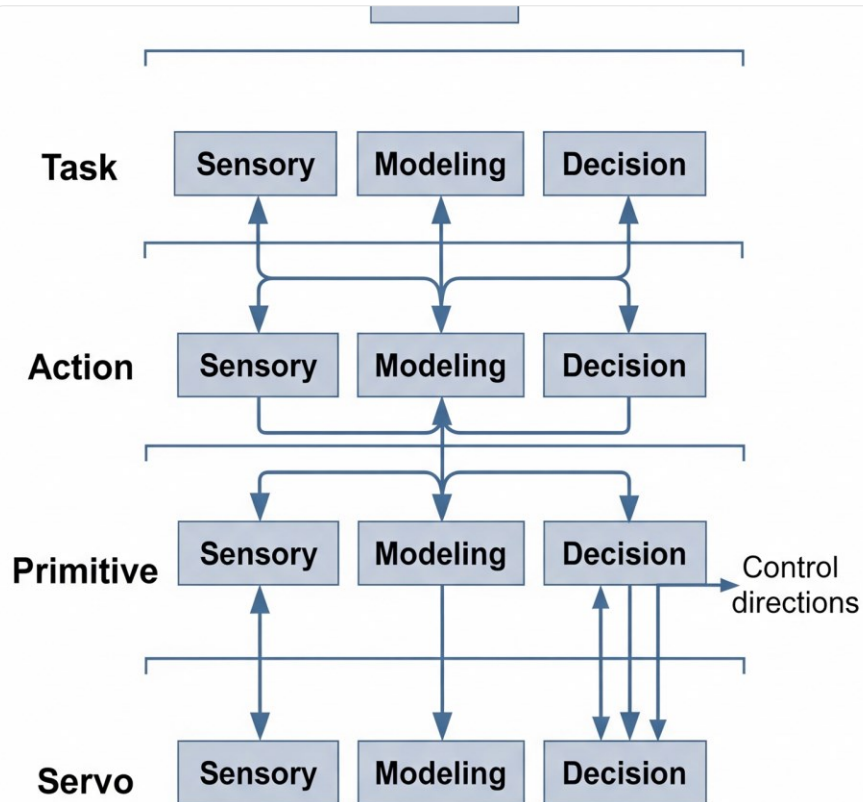
Mobile Robot Scenario:

Fleet Mgmt: Top-level site goals decomposed into individual robot task assignments.

CyberBot Scenario:

Logic: Multi-tiered architectural logic where UI, Middleware, and Database layers are isolated.

Hierarchical Functional Architecture Model



TASK LEVEL (Abstract Planning)

User specifies high-level goals. **Breakdown:** Task > Sequence of Actions. Updates via high-level vision.

ACTION LEVEL (Motion Path)

Commands translated to intermediate poses. **Check:** Obstacle avoidance & feasibility via geometric models.

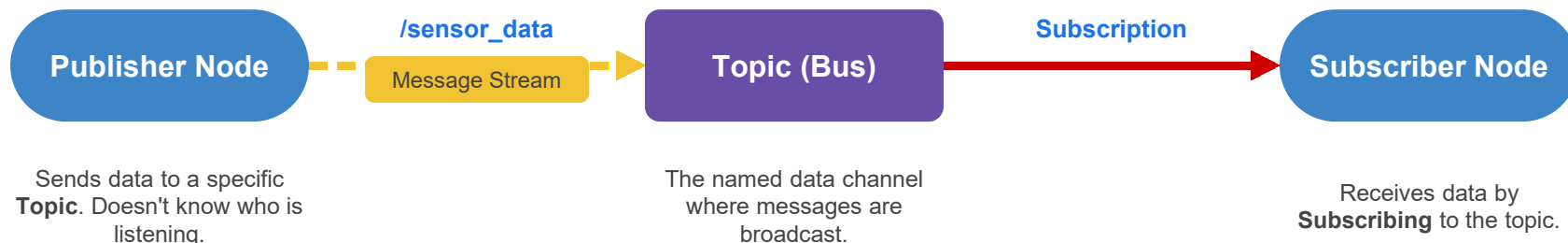
PRIMITIVE LEVEL (Trajectory)

Computation of motion trajectories & control strategy. Interpolates references for servo level.

SERVO LEVEL (Execution)

Real-time (ms) motor signals. Operates on error signals via proprioceptive sensors.

ROS2 Communication: The Publish-Subscribe Architecture



Key Concepts for Lab Implementation:

- **Anonymity:** Nodes communicate without knowing each other's identity, only the topic name.
- **Many-to-Many:** Multiple publishers can send to one topic, and multiple subscribers can receive from it.
- **ROS2 Jazzy Setup:** Use `ros2 topic echo` to monitor data streams during webinar exercises.
- **Quality of Service (QoS):** Policies that define how data is handled (reliability, durability, etc.).



Robot Programming Environments

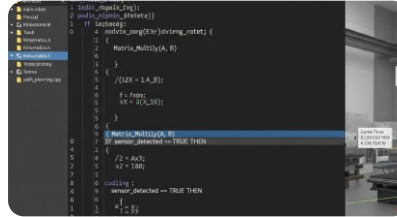


GEN 1: TEACHING-BY-SHOWING



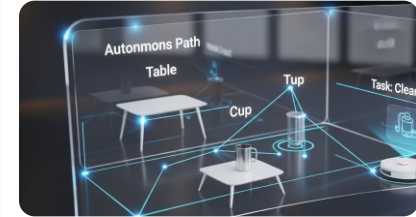
Primitive Level Access

GEN 2: ROBOT-ORIENTED



Action Level Access

GEN 3: OBJECT-ORIENTED



Task Level Access

Programming Paradigms

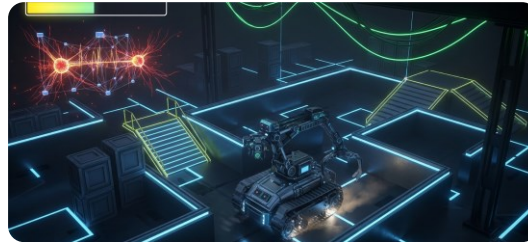
LEARNING FROM DEMONSTRATION



Imitation Learning

Robot learns complex behaviors by observing and mimicking human experts instead of manual coding.

DEEP REINFORCEMENT LEARNING



Trial-and-Error Optimization

Agent interacts with a simulated environment to maximize cumulative rewards for path planning.

EVOLUTIONARY & SWARM



Bio-Inspired Metaheuristics

Applying Genetic Algorithms and Swarm Intelligence for decentralized path planning and coordination.

Mapping

Map Paradigms: Location-Based vs. Feature-Based Maps

LOCATION-BASED MAPS (VOLUMETRIC)

Represent environment as a continuum of coordinates. **"What is at this specific spot?"** (e.g., occupancy grids).



Domestic & Industrial

Domestic: 2D/3D grids for floor/counters.

Warehouse: High-res metric grids for fixed conveyors & pallet zones.



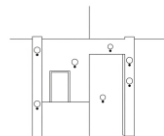
Specialized Domains

Cyber: IP/subnet grids tracking memory.

Deep Sea: 3D bathymetric elevation profile.

FEATURE-BASED MAPS (LANDMARK)

Extract landmark primitives (corners, walls, signatures). **"Where is this specific object/node?"**



Domestic & Industrial

Domestic: Sparse landmarks (table legs).

Warehouse: AprilTags, light patterns, reflective tape markers.



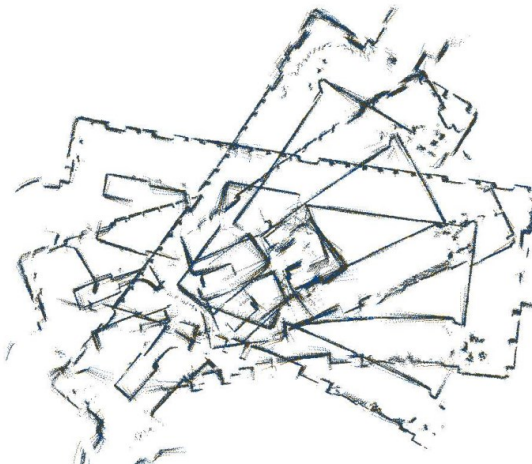
Specialized Domains

Cyber: Graph of servers and API endpoints.

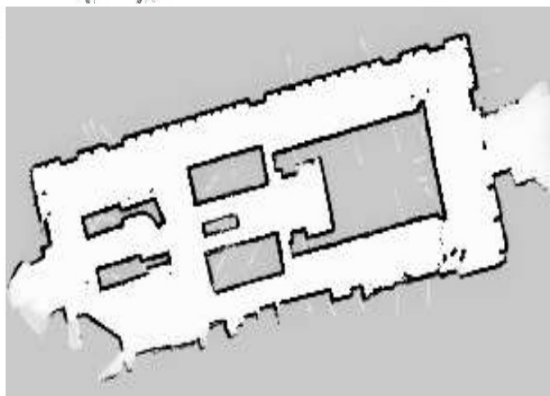
Deep Sea: Point-cloud of vent/pipe joints.

Mapping Problem

(a)



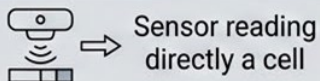
(b)



(a) Raw range data, position indexed by odometry. (b) Map

Inverse vs Forward Model

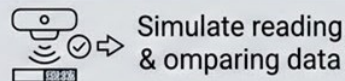
TRADITIONAL PER-GRID ESTIMATION (INVERSE MODEL)



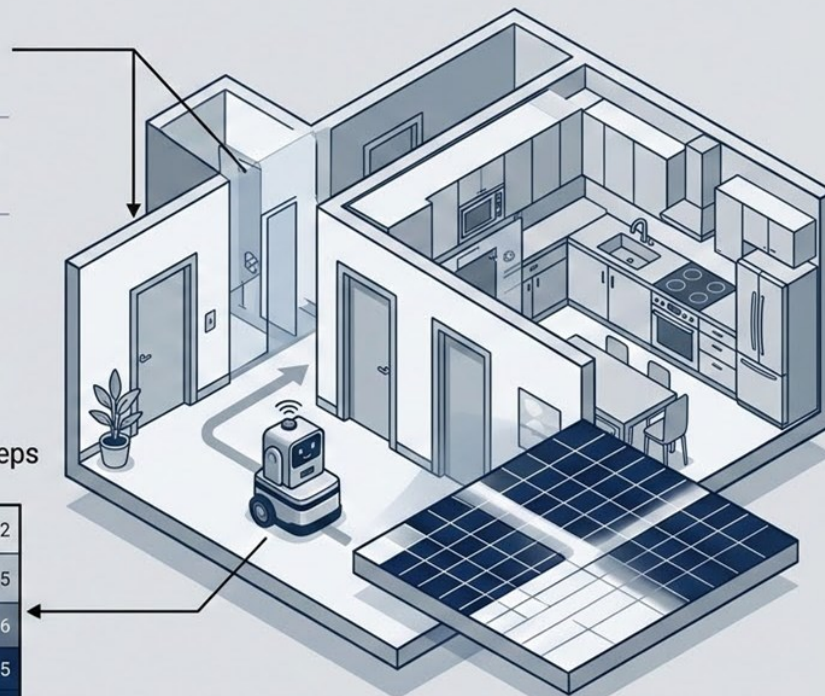
- ☐ $P(m_i|z_t, x_t)$
- ☐ Local Cell Independence Assumption
- ☐ Efficient Pixel-by-Pixel Updates
- ☐ Learns with BCE Loss



MAP & OPTIMIZATION METHODS (FORWARD MODEL)



- ☐ $P(z|m, x)$
- ☐ Physics-Based Sensor Simulation
- ☐ Objective Maximization (MAP)
- ☐ Hill-Climbing Algorithm Steps



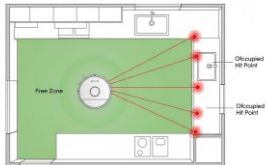
Mapping Elements

innovate

achieve

lead

1. Domestic Robot



Pose (X):
Kitchen floor (x,y,θ) .

Measurement (Z):
2D LiDAR / RGB-D.

Inverse Model:
Rangefinder logic: Pre-hit is free, Hit is occupied.

2. Warehouse AMR



Pose (X):
IMU/Encoder dead reckoning.

Measurement (Z):
Safety Scanners / ToF.

Inverse Model:
Stricter safety bounds; high penalty for dynamic noise.

3. Cyber Bot

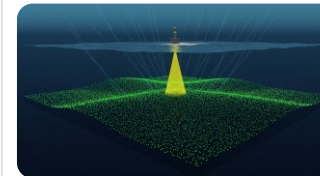


Pose (X):
Network depth / Stack pointer.

Measurement (Z):
Packets / Port Scans.

Inverse Model:
SYN-ACK = Occupied;
Timeout = Free.

4. Deep Sea AUV

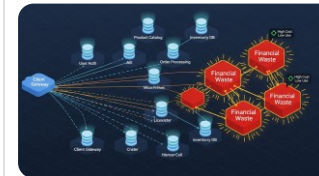


Pose (X):
3D 6-DOF (DVL/Depth).

Measurement (Z):
Multibeam Sonar.

Inverse Model:
Acoustic reflection; voxels marked as water or seabed.

5. FinOps Bot



Pose (X):
Service dependency path.

Measurement (Z):
Telemetry / Billing logs.

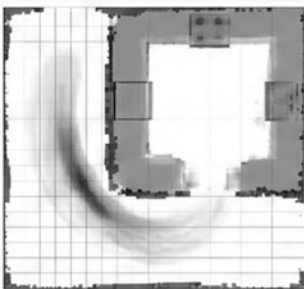
Inverse Model:
Low CPU + High Cost = Occupied (Waste).

Mapping Algorithms



Traditional Per-Grid Estimation (Bayesian)

- ✓ Updates each cell independently
- ✓ Log-odds recursive formula for recursive Bayes

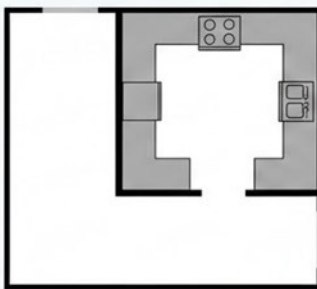


Traditional Hallway Mapping



Maximum A Posteriori (MAP) Optimization

- 🌐 Maximizes entire joint posterior $P(m|Z, X)$
- 🌐 Solves for globally consistent map m^*



MAP Optimized Grid



Hill-Climbing Optimization (with MAP)

- 🔍 Iterative heuristic search on a guess map
- 🔍 Active subset search only on line-of-sight cells
- 🔍 Mobile robot 'flipping' only illuminated cells from free to occupied



Hill-Climbing Local Update

Occupancy Grid Mapping

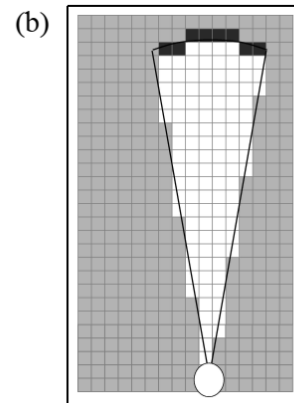
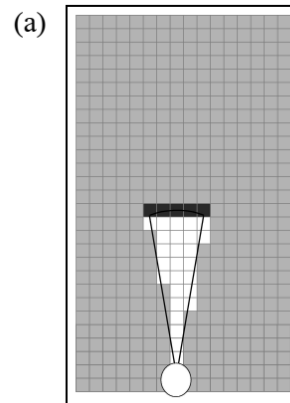
Occupancy grid mapping

OCCUPANCY GRID MAPPING- ALGORITHM



```
1:  Algorithm occupancy_grid_mapping( $\{l_{t-1,i}\}, x_t, z_t$ ):  
2:      for all cells  $m_i$  do  
3:          if  $m_i$  in perceptual field of  $z_t$  then  
4:               $l_{t,i} = l_{t-1,i} + \text{inverse\_sensor\_model}(m_i, x_t, z_t) - l_0$   
5:          else  
6:               $l_{t,i} = l_{t-1,i}$   
7:          endif  
8:      endfor  
9:      return  $\{l_{t,i}\}$ 
```

The algorithm occupancy grid mapping loops through all grid cells i , and updates those that fall into the sensor cone of the measurement z_t . For those where it does, it updates the occupancy value by virtue of the function inverse sensor model



Occupancy Grid Mapping: $P(m|Z, X)$

STEP 1: INITIALIZE

Set prior log-odds for cells m_i .

$$l_{0,i} = 0$$

(Assumes $p=0.5$)

STEP 2: RAY CASTING

- Project sensor beam z_t at pose x_t .
- Identify cells m_i in focus.

STEP 3: INVERSE MODEL

Calculate $p(m_i|z_t, x_t)$:

Before hit → **Free**
At hit → **Occupied**

STEP 4: RECURSIVE UPDATE

Update matrix $l_{t,i}$:

$$l_{t,i} = l_{t-1,i} + \text{inv} - l_{0,i}$$



PER-GRID ESTIMATION (Traditional Bayesian)

- Local Update (Individual Cells)
- Log-Odds Recursive
- Assumes Indep. Cells



Incremental local probability update



Bayesian posterior maximization per cell

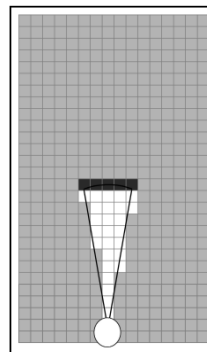


Sensor inverse model
 $l_{t,i} = l_{t-1,i} + \text{inv} - l_{0,i}$

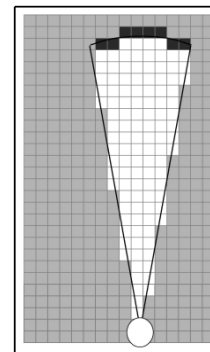


Low latency; linear complexity $O(N_{\text{cells}})$

(a)



(b)



DOMESTIC ROBOT

Pose: Floor (x,y,θ)

Z: LiDAR / RGB-D

Model: Rangefinder

WAREHOUSE AMR

Pose: High-res IMU/Encoder

Z: Safety Scanners/ToF

Model: High penalty for noise

CYBERSECURITY BOT

Pose: Network Depth/Stack

Z: Packets / Port Scans

Model: SYN-ACK = Occupied

Updating a 4x4 Occupancy Grid

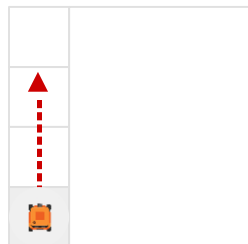


1. Initialize (Log-Odds)

0	0	0	0
0	0	0	0
0	0	0	0
0	0	0	0

Prior log-odds $l_0 = 0$ for all cells ($p = 0.5$ unknown).

2. Ray Casting & Inverse Model



Robot at (3,0) detects obstacle at (1,0).

- **Hit (1,0):** +1.39
- **Free (2,0):** -1.39

3. Recursive Update (l_t)

0	0	0	0
+1.39	0	0	0
-1.39	0	0	0
0	0	0	0

$$l_t = l_{t-1} + \text{Update} - l_0$$

Scores are simply summed into the grid matrix.

4. Probabilities (Final Map)

50%	50%	50%	50%
80%	50%	50%	50%
20%	50%	50%	50%
50%	50%	50%	50%

Converted via Sigmoid function.
80% confidence of obstacle at (1,0).
20% confidence of free space at (2,0).

Key Formulas used:

Log-Odds Update: $l(p) = \log(p / (1-p))$

Probability Recovery: $p = 1 - (1 / (1 + \exp(l_t)))$

Note: Summing log-odds is mathematically equivalent to Bayesian multiplication of probabilities but significantly more efficient for robotic hardware.

Occupancy grid mapping



A mobile robot is in a **2×2 grid environment**. The grid cells are numbered as follows:

```
+---+---+
| C1 | C2 |
+---+---+
| C3 | C4 |
+---+---+
```

Initially, the robot has **no knowledge** of obstacles, so it assigns to each cell a prior **occupancy probability** of **$P(\text{Occupied}) = 0.5$** . Now, the robot takes a sensor measurement. It detects an obstacle in cell **C1**. The sensor characteristics are: **$P(\text{Detect} | \text{Occupied}) = 0.8$** , **$P(\text{Detect} | \text{Free}) = 0.2$**

Update the occupancy probability for cell C1 using log-odds

Occupancy grid mapping



1. Compute Prior Log-Odds

First, convert the prior probability into **log-odds**:

$$L_{prior} = \log \frac{P(o)}{1-P(o)}$$

$$L_{prior} = \log \frac{0.5}{0.5} = \log(1) = 0 ; L_{prior} = 0$$

2. Compute Measurement Log-Odds

Next, compute the log-odds contribution from the measurement:

$$L_{measurement} = \log \frac{P(Detect|o)}{P(Detect|F)} = \log \frac{0.8}{0.2} = \log(4) \approx 1.386$$

3. Update Log-Odds

The updated log-odds: $L_{updated} = L_{prior} + L_{measurement} = 0 + 1.386 = 1.386$

4. Convert Log-Odds Back to Probability: Finally, convert the log-odds back into probability: $P(o) = \frac{1}{1 + e^{-L_{updated}}}$

$$= \frac{1}{1 + e^{-1.386}} \approx \frac{1}{1 + 0.25} = 0.8$$

The new occupancy probability of cell C1 is 0.8.

Occupancy grid mapping



- Solving through Inverse model

$$P(O | \text{Detect}) = \frac{P(\text{Detect} | O) \cdot P(O)}{P(\text{Detect})}$$

$$L_{inv} = \log \left(\frac{P(O | \text{Detect})}{1 - P(O | \text{Detect})} \right) = \log \left(\frac{0.8}{0.2} \right) = \log(4) = 1.386$$

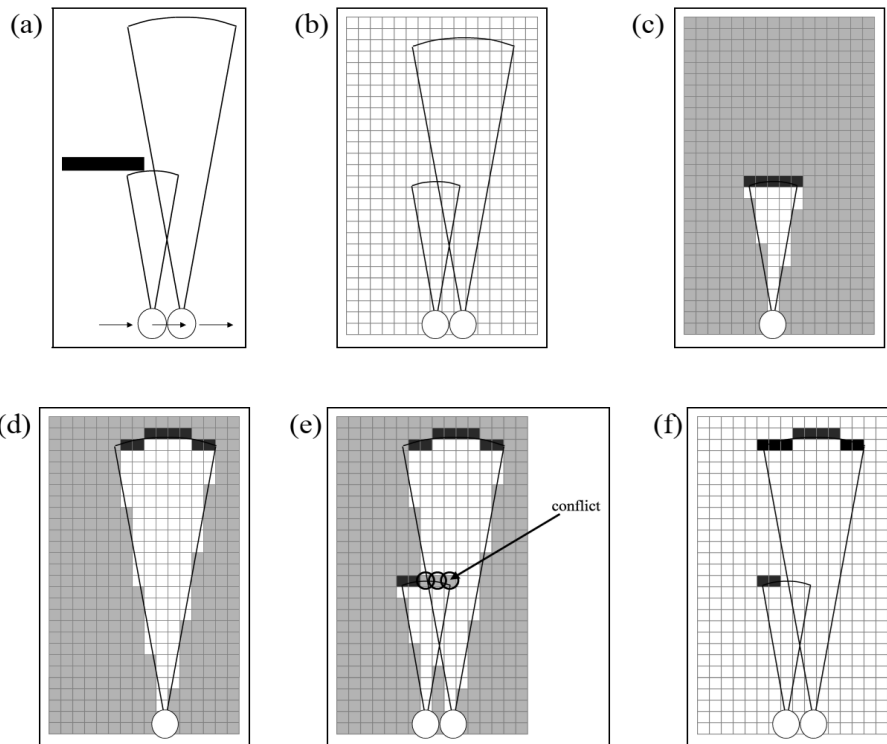
$$L_{updated} = L_{prior} + L_{inv} = 0 + 1.386 = 1.386$$

Why inverse :

- **Forward model** $\rightarrow P(\text{Detect} | \text{Occupied})$
- **Inverse model** $\rightarrow P(\text{Occupied} | \text{Detect}) =$ Given my measurement, how likely is the cell to be occupied?

$P(\text{Occupied} | \text{Detect}) = 0.8$ becomes direct update to map

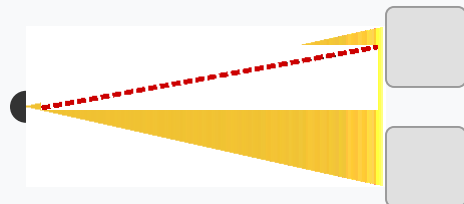
Occupancy grid mapping



The problem with the standard occupancy grid mapping algorithm : For the environment shown in Figure (a), a passing robot might receive the (noise-free) measurement shown in (b). The factorial approach maps these beams into probabilistic maps separately for each grid cell and each beam, as shown in (c) and (d). Combining both interpretations yields the map shown in (e). Obviously, there is a conflict in the overlap region, indicated by the circles in (e). The interesting insight is: There exist maps, such as the one in diagram (f), which perfectly explain the sensor measurement without any such conflict. For a sensor reading to be explained, it suffices to assume an obstacle somewhere in the cone of a measurement, and not everywhere.

Mapping Challenge A: The Data Association Problem

THE FAILURE MODE



Aliasing & Misassociation

Robot confuses **Chair A** with **Chair B** due to pose drift. Sonar/LiDAR "hits" land on the wrong historical feature.

THE MITIGATION SOLUTION

Validation Gates & Mahalanobis Distance

- Reject matches where distance $d >$ threshold.
- Semantic Anchors:** Fuse geometric data with unique IDs (e.g., Aisle Tags or MAC Addresses).
- Probabilistic Weights:** Update belief scores $P(z|m)$ instead of binary assignment.

"Mitigates map corruption by ensuring only high-confidence matches trigger a recursive update."

Domestic: Identical Chairs

Small slippage causes scans to smear across identical geometry.

Warehouse: Aisle Symmetry

Repetitive racking leads to catastrophic loop-closure failures.

Cyber: DHCP Reuse

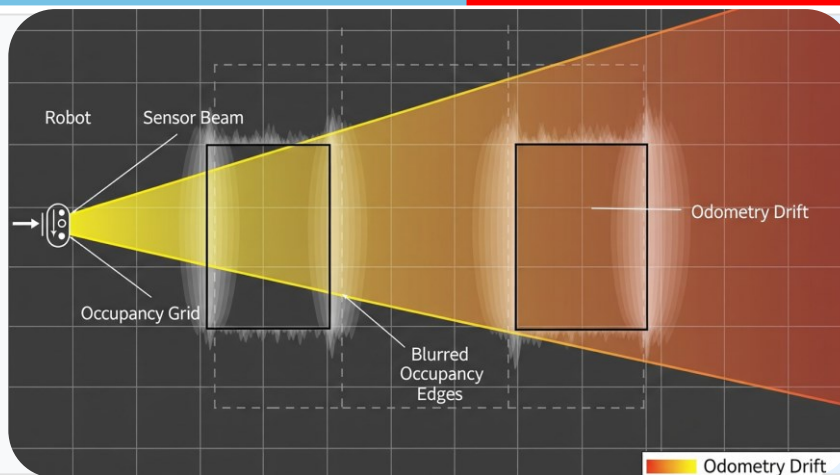
IP-based tracking misassociates vulnerabilities after ID recycle.

Mapping Challenge B: The Residual Uncertainty Problem

RESIDUAL DRIFT EFFECT

Edge Smearing

Accumulated noise and tracking drift persist even after filters, causing "fuzzy" occupancy boundaries.



MITIGATION STRATEGY

1. Covariance Inflation

Modeling unmeasured drift (slippage, payload shifts) as increased state uncertainty Σ_t .

2. Loop Closure Optimization

Global bundle adjustment to "snap" smeared features back to geometric truth.

3. Temporal Smoothing

Asynchronous logging alignment to fix latency jitter in FinOps/Cyber bots.

Domestic: Rug Transitions

Odometry errors from slick tiles and thick rugs smear occupancy grid edges over time.

Warehouse: Payload Shift

Weight changes alter wheel compression, creating systematic drift over 10-hour shifts.

Cyber: Latency Jitter

Dropped packets introduce timing uncertainty between probe and state recording.

FinOps : Telemetry Latency

Incorrect resource consumption data is logged. Microsecond level traffic spikes not recorded.

Occupancy Grid Numerical Example – Student Practice

Robot pose = $\{0,0,0^\circ\}$

Grid Resolution : 1m X1m cells

Sensor Reading : Z=5m along robot's heading 0°

Sensor FOV : $\pm 5^\circ$

Inverse Sensor model Parameters :

- $\alpha = 0.5\text{m}$
- Occupied Probability = 0.7
- Free Probability = 0.3
- Prior Probability= 0.5

Examine the 2D grid around the robot and update the occupancy probability.

Step 1: Candidate Grid cells

Step 2: Check Angular Constraint (β)

Step 3: Measure occupied/free distance

Step 4: Compute Log-odds updates

Step 5: Updated Grid